



# CS-250: Lab 04

## Runtime Complexity Analysis

**Muhammad Mujtaba**

**CMD ID: 540040**

[mmujtaba.bese25seecs@seecs.edu.pk](mailto:mmujtaba.bese25seecs@seecs.edu.pk)

**Class: BESE 16B**

**Batch: 2k25**

# Task 1:

Time Harness:

```
#include <chrono>
#include <print>

int main()
{
    int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    std::chrono::time_point start = std::chrono::steady_clock::now();

    //////////CODE////////CODE////////CODE////////CODE////////CODE////////CODE////////CODE////////
    //////////CODE////////CODE////////CODE////////CODE////////CODE////////CODE////////CODE////////
    //////////CODE////////CODE////////CODE////////CODE////////CODE////////CODE////////CODE////////

    std::chrono::time_point end = std::chrono::steady_clock::now();

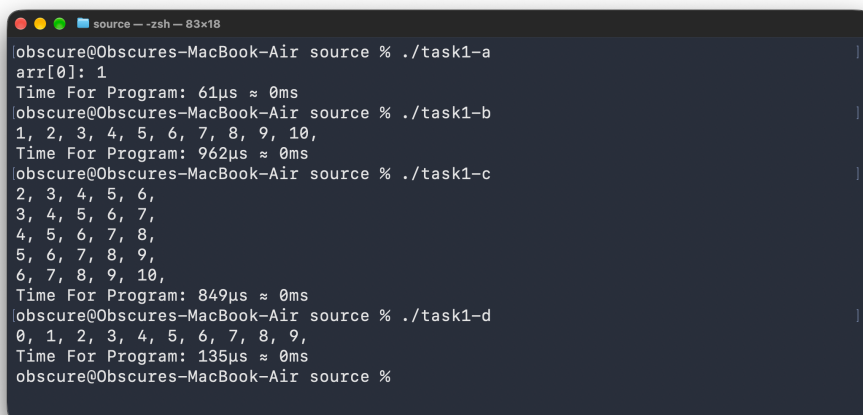
    auto timeInUs = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
    auto timeInMs = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::println("Time For Program: {} ≈ {}", timeInUs, timeInMs);

    return 0;
}
```

| Code | Complexity | Reason                                                             |
|------|------------|--------------------------------------------------------------------|
| A    | $O(1)$     | Equivalent to resolving a pointer address with pointer arithmetic. |
| B    | $O(N)$     | Looping over each element once.                                    |
| C    | $O(N^2)$   | Nested for loops, with inner one looping over the elements again.  |
| D    | $O(N)$     | Linearly scales with the number of elements in the linked list.    |

## Screenshots of runtime



```
source -- zsh -- 83x18
obscure@Obscures-MacBook-Air source % ./task1-a
arr[0]: 1
Time For Program: 61µs ≈ 0ms
obscure@Obscures-MacBook-Air source % ./task1-b
1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
Time For Program: 962µs ≈ 0ms
obscure@Obscures-MacBook-Air source % ./task1-c
2, 3, 4, 5, 6,
3, 4, 5, 6, 7,
4, 5, 6, 7, 8,
5, 6, 7, 8, 9,
6, 7, 8, 9, 10,
Time For Program: 849µs ≈ 0ms
obscure@Obscures-MacBook-Air source % ./task1-d
0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
Time For Program: 135µs ≈ 0ms
obscure@Obscures-MacBook-Air source %
```

## Task 2:

```
#include <chrono>
#include <print>
#include <random>
#include <optional>

#include "linked_list.hpp"

struct ParsedArgs
{
    int n = 10;
};

bool parseArgs(int argc, char *argv[], ParsedArgs *returnValue)
{
    if (argc < 2)
        return false;

    try
    {
        returnValue->n = std::stoi(argv[1]);
        return true;
    }
    catch (const std::exception &)
    {
        return false;
    }
}

#define SHOW_TIME(FUNC) \
do \
{ \
    std::chrono::time_point start = std::chrono::steady_clock::now(); \
    FUNC \
    std::chrono::time_point end = std::chrono::steady_clock::now(); \
    auto timeInUs = std::chrono::duration_cast<std::chrono::microseconds>(end - start); \
    auto timeInMs = std::chrono::duration_cast<std::chrono::milliseconds>(end - start); \
    std::println("Time For Program: {} ~ {}", timeInUs, timeInMs); \
} while (0);

int main(int argc, char *argv[])
{
    ParsedArgs args;
    parseArgs(argc, argv, &args);

    LinkedList myLinkedList{};
    for (size_t i = 0; i < args.n; i++)
    {
        myLinkedList.insertAt(std::rand() % 20, 0);
    }

    SHOW_TIME(
        int index = 0;
        size_t nodes_examined = 0;
        std::println("@index{{{}}}: {}; visited {} nodes", index, *myLinkedList.get(index,
&nodes_examined), nodes_examined); //
    );

    SHOW_TIME(
        int index = myLinkedList.count() / 2;
        size_t nodes_examined = 0;
        std::println("@index{{{}}}: {}; visited {} nodes", index, *myLinkedList.get(index,
&nodes_examined), nodes_examined); //
    );

    SHOW_TIME(
        int index = myLinkedList.count() - 1;
        size_t nodes_examined = 0;
        std::println("@index{{{}}}: {}; visited {} nodes", index, *myLinkedList.get(index,
&nodes_examined), nodes_examined); //
    );

    return 0;
}
```

|         | First |        |      | Middle |        |       | Last  |         |        |
|---------|-------|--------|------|--------|--------|-------|-------|---------|--------|
| N       | Big-O | Visits | Time | Big-O  | Visits | Time  | Big-O | Visits  | Time   |
| 100     | O(1)  | 1      | 68μs | O(N)   | 51     | 9μs   | O(N)  | 100     | 9μs    |
| 1,000   | O(1)  | 1      | 54μs | O(N)   | 501    | 30μs  | O(N)  | 1,000   | 43μs   |
| 10,000  | O(1)  | 1      | 74μs | O(N)   | 5,001  | 178μs | O(N)  | 10,000  | 322μs  |
| 100,000 | O(1)  | 1      | 77μs | O(N)   | 50,001 | 917μs | O(N)  | 100,000 | 1818μs |

**Note:** The reason the first access time are sometimes larger is due to data not being loaded into cache etc. Furthermore, calculations at microseconds scale can lead to noise. Time for println functions was also included in recorded time as well.

### Screenshots of runtime

```

obscur@Obscure-MacBook-Air bin % ./task-2 100
@index{0}: 4; visited 1 nodes
Time For Program: 68μs ≈ 0ms
@index{50}: 17; visited 51 nodes
Time For Program: 9μs ≈ 0ms
@index{99}: 7; visited 100 nodes
Time For Program: 9μs ≈ 0ms
obscur@Obscure-MacBook-Air bin % ./task-2 1000
@index{0}: 10; visited 1 nodes
Time For Program: 54μs ≈ 0ms
@index{500}: 1; visited 501 nodes
Time For Program: 30μs ≈ 0ms
@index{999}: 7; visited 1000 nodes
Time For Program: 43μs ≈ 0ms
obscur@Obscure-MacBook-Air bin % ./task-2 10000
@index{0}: 5; visited 1 nodes
Time For Program: 74μs ≈ 0ms
@index{5000}: 7; visited 5001 nodes
Time For Program: 178μs ≈ 0ms
@index{9999}: 7; visited 10000 nodes
Time For Program: 322μs ≈ 0ms
obscur@Obscure-MacBook-Air bin % ./task-2 100000
@index{0}: 14; visited 1 nodes
Time For Program: 77μs ≈ 0ms
@index{50000}: 11; visited 50001 nodes
Time For Program: 917μs ≈ 0ms
@index{99999}: 7; visited 100000 nodes
Time For Program: 1818μs ≈ 1ms
obscur@Obscure-MacBook-Air bin %

```

## Task 3:

```
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <print>
#include <string>

#include "linked_list.hpp"

constexpr size_t N = 100'000;

void accessFirst(int *array, LinkedList &list);
void accessLast(int *array, LinkedList &list);
void accessRandom(int *array, LinkedList &list);
void findValue(int *array, LinkedList &list, int value);

#define SHOW_TIME(FUNC) \
do \
{ \
    std::chrono::time_point start = std::chrono::steady_clock::now(); \
    FUNC \
    std::chrono::time_point end = std::chrono::steady_clock::now(); \
    auto timeInUs = std::chrono::duration_cast<std::chrono::microseconds>(end - start); \
    auto timeInMs = std::chrono::duration_cast<std::chrono::milliseconds>(end - start); \
    std::println("Time For Program: {} ≈ {}", timeInUs, timeInMs); \
} while (0);

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        std::println("Usage: ./task-3 --first | --last | --random | --find <value>");
        return 1;
    }

    // seed rand
    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    int *array = new int[N];
    LinkedList list{};

    for (size_t i = N; i > 0; i--)
    {
        const int value = std::rand() % 20;
        array[i - 1] = value;
        // list inserts in reverse order
        // 4, 3, 2, 1
        list.insertStart(value);
    }

    std::string operation = argv[1];

    if (operation == "--first")
        accessFirst(array, list);
    else if (operation == "--last")
        accessLast(array, list);
    else if (operation == "--random")
        accessRandom(array, list);
    else if (operation == "--find" && argc >= 3)
        findValue(array, list, std::atoi(argv[2]));
    else
        std::println("Invalid arguments");

    delete[] array;
    return 0;
}
```

```

void accessFirst(int *array, LinkedList &list)
{
    SHOW_TIME(
        std::println("Dynamic Array:");
        int value = array[0];
        std::println("Value: {}", value); //
    );

    SHOW_TIME(
        size_t examined = 0;
        int value = *list.get(0, &examined);
        std::println("Singly Linked List:");
        std::println("Value: {}; visited {} nodes", value, examined); //
    );
}

void accessLast(int *array, LinkedList &list)
{
    SHOW_TIME(
        std::println("Dynamic Array:");
        int value = array[N - 1];
        std::println("Value: {}", value); //
    );

    SHOW_TIME(
        size_t examined = 0;
        int value = *list.get(N - 1, &examined);
        std::println("Singly Linked List:");
        std::println("Value: {}; visited {} nodes", value, examined); //
    );
}

void accessRandom(int *array, LinkedList &list)
{
    size_t index = std::rand() % N;

    SHOW_TIME(
        std::println("Dynamic Array:");
        int value = array[index];
        std::println("Value: {}", value); //
    );

    SHOW_TIME(
        size_t examined = 0;
        int value = *list.get(index, &examined);
        std::println("Singly Linked List:");
        std::println("Value: {}; visited {} nodes", value, examined); //
    );
}

void findValue(int *array, LinkedList &list, int value)
{
    SHOW_TIME(
        int arrayIndex = -1;
        std::println("Dynamic Array:");
        for (size_t i = 0; i < N; i++) {
            if (array[i] == value) {
                arrayIndex = static_cast<int>(i);
                break;
            }
        } //
        std::println("Found {} at index {}", value, arrayIndex); //
    );

    SHOW_TIME(
        int listIndex = -1;
        std::println("Singly Linked List:");
        list.forEach([&](ConstNode node, size_t index) { //
            if (node->data == value) {
                listIndex = static_cast<int>(index);
                return true;
            }
            return false;
        });
        std::println("Found {} at index {}", value, listIndex); //
    );
}

```

| Operation | Dynamic Array |      | Linked List |        |
|-----------|---------------|------|-------------|--------|
|           | Complexity    | Time | Complexity  | Time   |
| First     | O(1)          | 64μs | O(1)        | 16μs   |
| Random    | O(1)          | 57μs | O(N)        | 1586μs |
| Last      | O(1)          | 85μs | O(N)        | 1644μs |
| Find      | O(N)          | 69μs | O(N)        | 12μs   |

**Note:** The reason the first access time are sometimes larger is due to data not being loaded into cache etc. Furthermore, calculations at microseconds scale can lead to noise. Time for println functions was also included in recorded time as well.

### Screenshots of runtime

```

obscure@Obscures-MacBook-Air bin % ./task-3 --first
Dynamic Array:
Value: 18
Time For Program: 64μs ≈ 0ms
Singly Linked List:
Value: 18; visited 1 nodes
Time For Program: 16μs ≈ 0ms
obscure@Obscures-MacBook-Air bin % ./task-3 --random
Dynamic Array:
Value: 10
Time For Program: 57μs ≈ 0ms
Singly Linked List:
Value: 10; visited 87399 nodes
Time For Program: 1586μs ≈ 1ms
obscure@Obscures-MacBook-Air bin % ./task-3 --last
Dynamic Array:
Value: 17
Time For Program: 85μs ≈ 0ms
Singly Linked List:
Value: 17; visited 100000 nodes
Time For Program: 1644μs ≈ 1ms
obscure@Obscures-MacBook-Air bin % ./task-3 --find 12
Dynamic Array:
Found 12 at index 6
Time For Program: 69μs ≈ 0ms
Singly Linked List:
Found 12 at index 6
Time For Program: 12μs ≈ 0ms
obscure@Obscures-MacBook-Air bin %

```

## Task 4:

```
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <print>

#include "linked_list.hpp"

constexpr size_t N = 100'000;

#define SHOW_TIME(FUNC) \
do \
{ \
    std::chrono::time_point start = std::chrono::steady_clock::now(); \
    FUNC \
    std::chrono::time_point end = std::chrono::steady_clock::now(); \
    auto timeInUs = std::chrono::duration_cast<std::chrono::microseconds>(end - start); \
    auto timeInMs = std::chrono::duration_cast<std::chrono::milliseconds>(end - start); \
    std::println("Time For Program: {} ~ {}", timeInUs, timeInMs); \
} while (0);

int main()
{
    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    LinkedList headOnlyList{};
    LinkedList pointerList{};

    for (size_t i = 0; i < N; i++)
    {
        const int value = std::rand() % 20;
        headOnlyList.insertStart(value);
        pointerList.insertStart(value);
    }

    SHOW_TIME(
        std::println("Delete last node using only head:");
        headOnlyList.removeLast(); //
    );

    // get pointers
    Node *previous = nullptr;
    Node *target = nullptr;
    pointerList.forEach([&](NodeLink current, size_t index) { //
        if (index == N - 2)
            previous = current;

        if (index == N - 1)
        {
            target = current;
            return true;
        }

        return false;
    });

    SHOW_TIME(
        std::println("Delete node with previous and target pointers:");
        previous->next = target->next;
        delete target; //
    );

    return 0;
}
```

|           | Linked List |              |
|-----------|-------------|--------------|
| Operation | Complexity  | Time         |
| Traversal | $O(N)$      | 2910 $\mu$ s |
| Pointer   | $O(1)$      | 8 $\mu$ s    |

### Screenshots of runtime

```

obscura@Obscure-MacBook-Air bin % ./task-4
Delete last node using only head:
Time For Program: 2910 $\mu$ s  $\approx$  2ms
Delete node with previous and target pointers:
Time For Program: 8 $\mu$ s  $\approx$  0ms
obscura@Obscure-MacBook-Air bin %

```

## Task 5:

```
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <print>
#include <string>

#include "linked_list.hpp"
#include "doubly_linked_list.hpp"

constexpr size_t N = 100'000;

void runSingly(LinkedList &list);
void runDoubly(DoublyLinkedList &list);

#define SHOW_TIME(...) \
do \
{ \
    std::chrono::time_point start = std::chrono::steady_clock::now(); \
    __VA_ARGS__ \
    std::chrono::time_point end = std::chrono::steady_clock::now(); \
    auto timeInUs = std::chrono::duration_cast<std::chrono::microseconds>(end - start); \
    auto timeInMs = std::chrono::duration_cast<std::chrono::milliseconds>(end - start); \
    std::println("Time For Program: {} ≈ {}", timeInUs, timeInMs); \
} while (0);

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        std::println("Usage: ./main --singly | --doubly");
        return 1;
    }

    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    LinkedList singly{};
    DoublyLinkedList doubly{};

    // same random integers in both lists
    for (size_t i = 0; i < N; i++)
    {
        const int value = std::rand() % 20;
        singly.insertStart(value);
        doubly.insertStart(value);
    }

    const std::string type = argv[1];

    if (type == "--singly")
        runSingly(singly);
    else if (type == "--doubly")
        runDoubly(doubly);
    else
    {
        std::println("Usage: ./main --singly | --doubly");
        return 1;
    }

    return 0;
}
```

```

void runSingly(LinkedList &list)
{
    Node *head = nullptr;
    Node *current = nullptr;

    // current pointer is already available before timing
    list.forEach([&](NodeLink node, size_t index) { //
        if (index == 0)
            head = node;

        if (index == N / 2)
        {
            current = node;
            return true;
        }

        return false;
    });

    SHOW_TIME(
        Node *next = current->next;
        std::println("Next: {}", next->data); //
    );

    SHOW_TIME(
        Node *previous = head;

        while (previous->next != current)
            previous = previous->next;

        std::println("Previous: {}", previous->data); //
    );

    const int insertAfterValue = std::rand() % 20;

    SHOW_TIME(
        Node *newNode = new Node{insertAfterValue, current->next};
        current->next = newNode;
        std::println("Inserted After: {}", insertAfterValue); //
    );

    const int insertBeforeValue = std::rand() % 20;

    SHOW_TIME(
        Node *previous = head;

        while (previous->next != current)
            previous = previous->next;

        previous->next = new Node{insertBeforeValue, current};
        std::println("Inserted Before: {}", insertBeforeValue); //
    );

    SHOW_TIME(
        Node *previous = head;

        while (previous->next != current)
            previous = previous->next;

        previous->next = current->next;
        delete current;
        current = nullptr;

        std::println("Deleted"); //
    );
}

```

```

void runDoubly(DoublyLinkedList &list)
{
    DoubleSidedNode *current = nullptr;

    // current pointer is already available before timing
    list.forEach([&](DoubleSidedNodeLink node, size_t index) { //
        if (index == N / 2)
        {
            current = node;
            return true;
        }

        return false;
    });

    SHOW_TIME(
        DoubleSidedNode *next = current->next;
        std::println("Next: {}", next->data); //
    );

    SHOW_TIME(
        DoubleSidedNode *previous = current->previous;
        std::println("Previous: {}", previous->data); //
    );

    const int insertAfterValue = std::rand() % 20;

    SHOW_TIME(
        DoubleSidedNode *newNode = new DoubleSidedNode{
            insertAfterValue,
            current,
            current->next};

        if (current->next != nullptr) current->next->previous = newNode;

        current->next = newNode;

        std::println("Inserted After: {}", insertAfterValue); //
    );

    const int insertBeforeValue = std::rand() % 20;

    SHOW_TIME(
        DoubleSidedNode *newNode = new DoubleSidedNode{
            insertBeforeValue,
            current->previous,
            current};

        current->previous->next = newNode; current->previous = newNode;

        std::println("Inserted Before: {}", insertBeforeValue); //
    );

    SHOW_TIME(
        current->previous->next = current->next;

        if (current->next != nullptr)
            current->next->previous = current->previous;

        delete current;
        current = nullptr;

        std::println("Deleted"); //
    );
}

```

| Operation     | Singly     |      | Double     |      |
|---------------|------------|------|------------|------|
|               | Complexity | Time | Complexity | Time |
| Next          | O(1)       | 73μs | O(1)       | 45μs |
| Previous      | O(N)       | 98μs | O(1)       | 6μs  |
| Insert After  | O(1)       | 4μs  | O(1)       | 5μs  |
| Insert Before | O(N)       | 95μs | O(1)       | 4μs  |
| Delete        | O(N)       | 97μs | O(1)       | 3μs  |

**Note:** The reason the first access time are sometimes larger is due to data not being loaded into cache etc. Furthermore, calculations at microseconds scale can lead to noise. *Time for println functions was also included in recorded time as well.*

### Screenshots of runtime

```

obscura@Obscure-MacBook-Air bin % ./task-5 --singly
Next: 13
Time For Program: 73μs ≈ 0ms
Previous: 18
Time For Program: 98μs ≈ 0ms
Inserted After: 12
Time For Program: 4μs ≈ 0ms
Inserted Before: 12
Time For Program: 95μs ≈ 0ms
Deleted
Time For Program: 97μs ≈ 0ms
obscura@Obscure-MacBook-Air bin % ./task-5 --doubly
Next: 3
Time For Program: 45μs ≈ 0ms
Previous: 11
Time For Program: 6μs ≈ 0ms
Inserted After: 13
Time For Program: 5μs ≈ 0ms
Inserted Before: 19
Time For Program: 4μs ≈ 0ms
Deleted
Time For Program: 3μs ≈ 0ms
obscura@Obscure-MacBook-Air bin %

```

**Final Question:** The doubly linked list trades the additional memory of storing the previous pointer to get the previous node in O(1) time complexity.

Examples:

- 1. Moving the Previous Node:** In singly linked list moving to the previous node requires traversal from the head to that point, while in doubly linked list, we can just use the pointer of that node
- 2. Deleting a Node:** To delete a node, the pointer references for previous and last node must be changed, doing this in singly linked list requires traversal from head as discussed, while in doubly linked list, we can use the previous pointers in the node.